

Data Structures and Algorithms – Lab Manual

LAB – 09

Stack Implementation via Arrays and Linked List

Lab Instructor: Muniba khan

Lab 9	
Contents:	Stacks implementation via arrays and linked list, Push, Pop, Is empty, Top function, Count and Stacks STL functions implementation

Objectives of the Lab

After completing this lab, YOU will be able to:

- Understand the concept of Stack (LIFO)
- Implement Stack using Arrays
- Implement Stack using Linked List
- Perform basic stack operations: Push, Pop, IsEmpty, Top, Count
- Use C++ STL stack library

Stack

A **Stack** is a linear data structure that follows the **LIFO (Last In First Out)** principle. The element inserted last is removed first.

Basic Stack Operations

- **Push:** Insert an element into stack
- **Pop:** Remove an element from stack
- **IsEmpty:** Check whether stack is empty
- **Top (Peek):** Return top element without removing it
- **Count:** Return total elements in stack

Q1. Stack Implementation Using Arrays

Problem Statement

Write a C++ program to implement stack using arrays and perform the following operations:

- Push
- Pop
- IsEmpty
- Top
- Count

```
#include <iostream>
using namespace std;

const int MAX = 5;    // Maximum size of stack

int stack[MAX];       // Stack array
int top = -1;         // Top pointer (stack is empty)

void push(int value) {
    if (top == MAX - 1)
```

```
        cout << "Stack Overflow" << endl;
    else {
        top++;
        stack[top] = value;
        cout << value << " pushed into stack" << endl;
    }
}

void pop() {
    if (top == -1)
        cout << "Stack Underflow" << endl;
    else {
        cout << stack[top] << " popped from stack" << endl;
        top--;
    }
}

bool isEmpty() {
    return (top == -1);
}

int getTop() {
    if (top == -1)
        return -1;
    else
        return stack[top];
}

int count() {
    return top + 1;
}

int main() {
    push(10);
    push(20);
    push(30);

    cout << "Top element: " << getTop() << endl;
    cout << "Total elements: " << count() << endl;
```

```

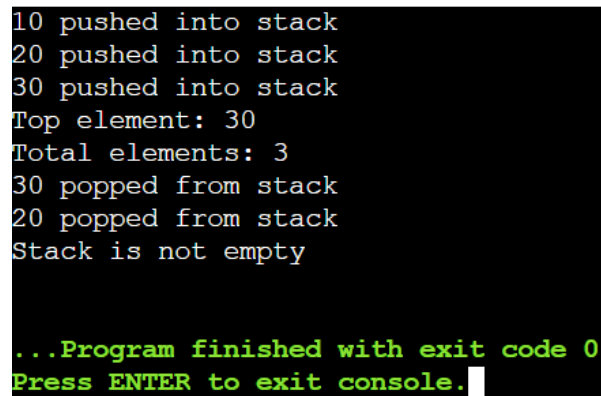
    pop();
    pop();

    if (isEmpty())
        cout << "Stack is empty" << endl;
    else
        cout << "Stack is not empty" << endl;

    return 0;
}

```

Output



```

10 pushed into stack
20 pushed into stack
30 pushed into stack
Top element: 30
Total elements: 3
30 popped from stack
20 popped from stack
Stack is not empty

...Program finished with exit code 0
Press ENTER to exit console.

```

Q2. Stack Implementation Using STL Library

Problem Statement

Write a C++ program to implement stack using **STL stack library** and perform push, pop, top, empty, and size operations.

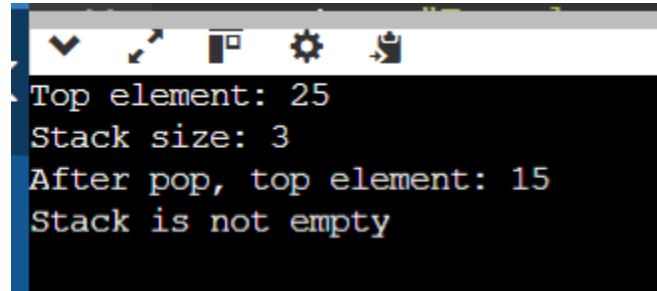
1. What is STL stack?

- **STL stack** is a **container adapter** provided by the C++ Standard Template Library (STL).
- It provides a **stack data structure** with standard operations:
 - push(x) → add element on top
 - pop() → remove top element
 - top() → access top element
 - empty() → check if stack is empty
 - size() → number of element

Solution (C++ Code)

```
1  #include <iostream>
2  #include <stack>
3  using namespace std;
4  int main()
5  {
6      stack<int> s;
7      s.push(5);
8      s.push(15);
9      s.push(25);
10
11     cout << "Top element: " << s.top() << endl;
12     cout << "Stack size: " << s.size() << endl;
13
14     s.pop();
15     cout << "After pop, top element: " << s.top() << endl;
16
17     if (s.empty())
18         cout << "Stack is empty" << endl;
19     else
20         cout << "Stack is not empty" << endl;
21
22     return 0;
23 }
```

Output



```
Top element: 25
Stack size: 3
After pop, top element: 15
Stack is not empty
```

Q3. Stack Implementation Using Linked List

Problem Statement

Write a C++ program to implement stack using linked list.

Instead of using arrays or STL stack, the stack should **dynamically grow and shrink** by adding or removing nodes in a linked list, following the **LIFO (Last In, First Out)** principle.

Solution (C++ Code)

```
#include <iostream>
using namespace std;

struct Node
{
    int data;
    Node* next;
};

Node* top = NULL;

void push(int value)
{
    Node* newNode = new Node();
    newNode->data = value;
    newNode->next = top;
    top = newNode;
    cout << value << " pushed into stack" << endl;
}

void pop() {
    if (top == NULL)
        cout << "Stack Underflow" << endl;
    else {
        Node* temp = top;
        cout << temp->data << " popped from stack" << endl;
        top = top->next;
        delete temp;
    }
}
```

```
void displayTop() {  
    if (top == NULL)  
        cout << "Stack is empty" << endl;  
    else  
        cout << "Top element: " << top->data << endl;  
}  
  
int main() {  
    push(100);  
    push(200);  
    push(300);  
  
    displayTop();  
  
    pop();  
    pop();  
  
    displayTop();  
  
    return 0;  
}
```

Output

```
100 pushed into stack  
200 pushed into stack  
300 pushed into stack  
Top element: 300  
300 popped from stack  
200 popped from stack  
Top element: 100
```

Lab Tasks

Instructions:

- Solve the following problems using C++.
- Think carefully about how the data should be stored and managed.
- You may implement any necessary functions to complete the tasks.

Q1: Browser History Simulation

A browser keeps track of pages visited. The user can go “Back” to the previous page.

- Simulate storing the last 5 pages visited.
- Go back 2 pages.
- Display the current page.
- Display the total number of pages remaining in history.

Q2: Stack Using Array

- Implement a stack of integers using an array.
- Perform 3 push operations, 2 pop operations.
- Display the top element and total elements in the stack.

Q3 (Scenario – Linked List Stack)

Scenario: Undo-Redo System in a Drawing App

A drawing application keeps track of user actions (like drawing lines, shapes, or erasing) for **Undo** functionality.

- Each action is stored as an integer ID.
- The last action performed can be undone first (LIFO).

Tasks:

1. Implement a stack using a **linked list** to store user actions.
2. Perform the following sequence:
 - Push 5 actions (IDs: 101, 102, 103, 104, 105).
 - Undo 3 actions (pop operations).
 - Display the most recent action still active.
 - Check if the stack is empty after the undo operations.
3. Add an extra check: If the stack becomes empty and the user tries to undo another action, display a “**No actions to undo**” message.

END